

The syntax for calling this version of `dropWhile` looks like `dropWhile(xs)(f)`. That is, `dropWhile(xs)` is returning a function, which we then call with the argument `f` (in other words, `dropWhile` is *curried*⁷). The main reason for grouping the arguments this way is to assist with type inference. We can now use `dropWhile` without annotations:

```
val xs: List[Int] = List(1,2,3,4,5)
val ex1 = dropWhile(xs)(x => x < 4)
```

Note that `x` is not annotated with its type.

More generally, when a function definition contains multiple argument groups, type information flows from left to right across these argument groups. Here, the first argument group fixes the type parameter `A` of `dropWhile` to `Int`, so the annotation on `x => x < 4` is not required.⁸

We'll often group and order our function arguments into multiple argument lists to maximize type inference.

3.4 *Recursion over lists and generalizing to higher-order functions*

Let's look again at the implementations of `sum` and `product`. We've simplified the `product` implementation slightly, so as not to include the “short-circuiting” logic of checking for `0.0`:

```
def sum(ints: List[Int]): Int = ints match {
  case Nil => 0
  case Cons(x,xs) => x + sum(xs)
}

def product(ds: List[Double]): Double = ds match {
  case Nil => 1.0
  case Cons(x, xs) => x * product(xs)
}
```

Note how similar these two definitions are. They're operating on different types (`List[Int]` versus `List[Double]`), but aside from this, the only differences are the value to return in the case that the list is empty (0 in the case of `sum`, 1.0 in the case of `product`), and the operation to combine results (+ in the case of `sum`, * in the case of `product`). Whenever you encounter duplication like this, you can generalize it away by pulling subexpressions out into function arguments. If a subexpression refers to any local variables (the + operation refers to the local variables `x` and `xs` introduced by the pattern, similarly for `product`), turn the subexpression into a function that accepts these variables as arguments. Let's do that now. Our function will take as arguments

⁷ Recall from the previous chapter that a function of two arguments can be represented as a function that accepts one argument and returns another function of one argument.

⁸ This is an unfortunate restriction of the Scala compiler; other functional languages like Haskell and OCaml provide *complete* inference, meaning type annotations are almost never required. See the notes for this chapter for more information and links to further reading.

the value to return in the case of the empty list, and the function to add an element to the result in the case of a nonempty list.⁹

Listing 3.2 Right folds and simple uses

```
def foldRight[A,B](as: List[A], z: B)(f: (A, B) => B): B =
  as match {
    case Nil => z
    case Cons(x, xs) => f(x, foldRight(xs, z)(f))
  }
```

```
def sum2(ns: List[Int]) =
  foldRight(ns, 0)((x,y) => x + y)
```

```
def product2(ns: List[Double]) =
  foldRight(ns, 1.0)(_ * _)
```

Again, placing `f` in its own argument group after `as` and `z` lets type inference determine the input types to `f`.

`_ * _` is more concise notation for `(x,y) => x * y` (see following sidebar).

`foldRight` is not specific to any one type of element, and we discover while generalizing that the value that's returned doesn't have to be of the same type as the elements of the list! One way of describing what `foldRight` does is that it replaces the constructors of the list, `Nil` and `Cons`, with `z` and `f`, illustrated here:

```
Cons(1, Cons(2, Nil))
f (1, f (2, z ))
```

Let's look at a complete example. We'll *trace* the evaluation of `foldRight(Cons(1, Cons(2, Cons(3, Nil))), 0)((x,y) => x + y)` by repeatedly substituting the definition of `foldRight` for its usages. We'll use program traces like this throughout this book:

```
foldRight(Cons(1, Cons(2, Cons(3, Nil))), 0)((x,y) => x + y)
1 + foldRight(Cons(2, Cons(3, Nil)), 0)((x,y) => x + y)
1 + (2 + foldRight(Cons(3, Nil), 0)((x,y) => x + y))
1 + (2 + (3 + (foldRight(Nil, 0)((x,y) => x + y))))
1 + (2 + (3 + (0)))
6
```

Replace `foldRight` with its definition.

Note that `foldRight` must traverse all the way to the end of the list (pushing frames onto the call stack as it goes) before it can begin collapsing it.

Underscore notation for anonymous functions

The anonymous function `(x,y) => x + y` can be written as `_ + _` in situations where the types of `x` and `y` could be inferred by Scala. This is a useful shorthand in cases where the function parameters are mentioned just once in the body of the function. Each underscore in an anonymous function expression like `_ + _` introduces a new (unnamed) function parameter and references it. Arguments are introduced in left-to-right order. Here are a few more examples:

⁹ In the Scala standard library, `foldRight` is a method on `List` and its arguments are curried similarly for better type inference.

(continued)

```

_ + _      ← (x,y) => x + y
_ * 2      ← x => x * 2
_.head     ← xs => xs.head
_ drop _   ← (xs,n) => xs.drop(n)

```

Use this syntax judiciously. The meaning of this syntax in expressions like `foo(_, g(List(_ + 1), _))` can be unclear. There are precise rules about scoping of these underscore-based anonymous functions in the Scala language specification, but if you have to think about it, we recommend just using ordinary named function parameters.

**EXERCISE 3.7**

Can `product`, implemented using `foldRight`, immediately halt the recursion and return `0.0` if it encounters a `0.0`? Why or why not? Consider how any short-circuiting might work if you call `foldRight` with a large list. This is a deeper question that we'll return to in chapter 5.

**EXERCISE 3.8**

See what happens when you pass `Nil` and `Cons` themselves to `foldRight`, like this: `foldRight(List(1,2,3), Nil:List[Int])(Cons(_,_)).10` What do you think this says about the relationship between `foldRight` and the data constructors of `List`?

**EXERCISE 3.9**

Compute the length of a list using `foldRight`.

```
def length[A](as: List[A]): Int
```

**EXERCISE 3.10**

Our implementation of `foldRight` is not tail-recursive and will result in a `StackOverflowError` for large lists (we say it's not *stack-safe*). Convince yourself that this is the case, and then write another general list-recursion function, `foldLeft`, that is

¹⁰ The type annotation `Nil:List[Int]` is needed here, because otherwise Scala infers the `B` type parameter in `foldRight` as `List[Nothing]`.